

INCIDENTE LOGIN · v2 · gap residual identificado

Owner: Celestino · 2026-05-02 · Hotfix v1 mergeado en main (commit **d9a6ea4**, PR #32) y desplegado · login del usuario **sigue fallando** · branch nueva **hotfix/auth-cross-customer-recovery**

Estado: Causa probable confirmada por revisión del código (sin Stripe API). Hay un gap en el matcher v1: el password del usuario coincide con un customer SIN sub válida, y el password del customer CON sub válida es distinto. Hotfix v2 implementado y testeado (15/15) en rama. **Recovery operacional disponible HOY** sin código nuevo · explicado abajo. **Cero writes** a Stripe/DB/env/WhatsApp/email.

1. Hotfix v1 está deployed

Verificación	Resultado
git log origin/main	d9a6ea4 Merge PR #32 (hotfix/auth-multi-customer-match)
GET https://usadona.com/login	200 · 0.29s
Mensaje en producción	El UI muestra el mensaje nuevo ("No pudimos validar...") → confirmado por el owner
Pero login del usuario afectado	sigue fallando

2. Causa probable · gap residual del matcher v1

El matcher v1 hace, simplificado:

```
for each customer in customers.list({email}):
  if password == derivePassword(customer.id):
    if customer has sub válida:
      return match
return null
```

Caso reportado en producción para *celestinojb@gmail.com*:

Customer	Email	Sub	Password derivado conocido por el usuario
cus_OLD (más antiguo)	celestinojb@gmail.com	canceled / sin sub válida	SÍ · le llegó welcome de éste
cus_NEW (con plan pago)	celestinojb@gmail.com	active	NO · su welcome nunca llegó al usuario, o fue silenciado

Resultado del matcher v1: itera cus_OLD, password match ✓, sub no válida → *continue*. Itera cus_NEW, password no match (HMAC con customer_id distinto produce hash distinto) → *continue*. **Retorna null**. El usuario ve mensaje de error aunque tenga plan pago y conozca un password legítimo.

3. Otras hipótesis descartadas o confirmadas

Hipótesis	Veredicto
Password de customer viejo	HIPÓTESIS PRINCIPAL · confirmada por análisis estático del código + flujo de welcome
Email distinto en Stripe (alias, mayúscula, punto o tilde)	Posible · descartar revisando manualmente customers en Stripe Dashboard. Stripe normaliza por email
Sub en estado no incluido (incomplete/unpaid/paused)	Posible · probable: el owner confirma que el plan está pago. Estados válidos: active, trialing, past_due.
DASHBOARD_PASSWORD_SECRET cambió en producción	Posible pero raro. Si pasó, TODOS los passwords previos quedan inválidos · revisar fecha del último commit
Bug del matcher v1	NO · 12/12 tests passing · lógica correcta para los casos cubiertos · gap es de cobertura

4. Recuperar acceso HOY · sin nuevo código

Procedimiento operacional para el owner. Usa el CLI helper de T1.4.B que ya existe en el repo. **NO requiere merge ni deploy.** NO comprar otro plan.

Paso 1 · Identificar customer con sub activa

```
# Con tu STRIPE_SECRET_KEY local (no se comparte)
stripe customers list \
  --email celestinojb@gmail.com \
  --limit 100 \
  --expand 'data.subscriptions'

# Buscar el cus_xxx cuyo subscriptions.data tenga
# status: active (o trialing / past_due).
```

Paso 2 · Derivar el password del customer correcto

```
# desde el directorio landing/
DASHBOARD_PASSWORD_SECRET=<mismo-valor-que-Vercel> \
  npx tsx scripts/derive-dashboard-password.ts cus_NEW_XXXXX

# Output: dona-XXXXXXXXXXXX (12 hex chars · NO loguear)
```

Paso 3 · Entregar al usuario por canal seguro

- El password sale por **stdout una sola vez**. NO se queda en logs ni en historial de scripts (el helper no usa console.log).
- Entregar por **WhatsApp directo** al teléfono registrado (canal cifrado E2E) · igual que se hacía pre-T2.0.B.
- **NO** mandar por email · NO escribir en Slack/Discord/ticket público · NO subir a Notion/GDrive sin cifrar.
- Una vez entregado, el usuario puede entrar al dashboard. El matcher v1 actual le dará acceso porque ese password coincide con cus_NEW que tiene la sub activa.

Riesgo si NO se aplica el hotfix v2: cualquier nuevo usuario que sufra el mismo escenario (welcome llegó del customer equivocado) tendrá que hacer este procedimiento manual. El v2 lo automatiza.

5. Hotfix v2 · recovery automático

Branch: `hotfix/auth-cross-customer-recovery` · base `main@d9a6ea4`. Implementado, testado, NO mergeado.

Cambio de lógica en lib/auth-matcher.ts

```
# Pass 1 · match exacto (igual que v1)
for each customer in customers.list({email}):
  if password == derive(customer.id):
    if customer has sub valida:
      return match
    else:
      passwordOwner = customer # ← nuevo · marca ownership

# Pass 2 · cross-customer recovery
if passwordOwner is None: return null
for each customer in same-email candidates:
  if customer.id == passwordOwner.id: continue
  if customer has sub valida:
    return { customer, sub, recovery: true,
            passwordOwnerCustomerId: passwordOwner.id }
return null
```

Por qué Pass 2 es seguro

- Pass 2 solo se activa si **ownership ya fue demostrado** por un password match en Pass 1.
- El password derivado de *cus_OLD* solo lo conoce quien recibió el welcome de ese customer · es prueba de posesión legítima del email.
- Stripe trata customers con mismo email como mismo cliente humano · autorizar el dashboard de la sub activa del mismo email es coherente con la mental model del usuario.
- **Riesgo residual aceptado:** si dos personas distintas comparten un email accidentalmente, una con password de su customer puede ver dashboard del otro. Probabilidad muy baja y consistente con el modelo Dona (1 email = 1 cliente).
- **Logging:** auth.ts loguea WARN sin PII cuando ocurre recovery (sub_id, customer_id **truncados**), para trackear el patrón en Vercel logs y motivar housekeeping de duplicados.

6. Tests · 15/15 passing

#	Test	Cubre
1	email sin customers → null	Sanity
2	1 customer con sub canceled → null	Política
3	2 customers, sub activa en 2do, password match cus_NEW → match cus_NEW · recovery=false	Casos NEW
4	Password incorrecto cus_1, correcto cus_2 con sub → cus_2	Iteración respeta password por customer
5	Sub trialing → autorizada	Política
6	Sub past_due → autorizada	Política
7	Sub incomplete → null	Política
8	Customer deleted=true → saltado	Robustez
9	DASHBOARD_PASSWORD_SECRET ausente → null	Sin path permisivo
10	Email vacío → null	Validación
11	Password vacío → null	Validación
12	ESTADOS_SUB_VALIDOS spec lock	Spec
13	RECOVERY · password viejo cus_OLD + sub en cus_NEW → match cus_NEW in recovery	Casos NEW
14	Sin ownership (password no match ningún customer) → null aunque sub activa	Seguridad sub activa sin ownership
15	Ownership demostrado pero NINGÚN customer del email tiene sub canceled en todos · respeta política	Casos sub canceled en todos

7. Validación local

Comando (cwd landing/)	Resultado
npm test	15/15 passing · 363ms
npx tsc --noEmit	0 errores
rm -rf .next && npm run build	OK · 17/17 pages
npm run lint	0 errors · 3 warnings preexistentes · exit 0

8. T2.0.B welcome · ¿se marcó al customer correcto?

Lectura del código de *agent/welcome.py* y *agent/billing.py*:

- El welcome se dispara solo en *accion='created'* del *checkout.session.completed*. **NO** en *updated* · NO en re-checkout.
- El password se deriva del *customer_id* del checkout que disparó el evento, no del cus más reciente del email.
- El flag *bienvenida_enviada* vive en *SuscripcionStripe.subscription_id*, NO en customer. Una sub nueva = welcome nuevo (correcto).
- **Implicación:** si el usuario hizo 2 checkouts (uno hace tiempo y uno reciente), el welcome reciente debió enviarse con password de cus_NEW. Si NO le llegó, posibles causas:
- Whapi falló silenciosamente (retorno False) → flag NO marcado · checkable con *SELECT bienvenida_enviada FROM suscripcion_stripe WHERE subscription_id=<sub_NEW>*.
- *WELCOME_DRY_RUN=true* en producción por error → flag se marca pero no se envía. Verificar en Vercel/Render env.
- Sub_NEW se creó **antes** de T2.0.B (~2 días) · si así, no había welcome automático y el owner debió enviar manual.
- Customer recibió welcome pero el celular cambió de número WhatsApp · llegó al teléfono viejo.

Sugerencia diagnóstica (read-only · al owner): consultar */admin/jobs-recientes?telefono=<num>* y la fila *SuscripcionStripe* del email. Si *bienvenida_enviada=true* pero al usuario no le llegó, regenerar con el helper CLI (sección 4).

9. Confirmaciones explícitas

NO se ejecutó nada de esto en esta sesión:

- Stripe writes (customers.update / subscriptions.cancel / refund / delete)
- DB writes en backend Render (ni siquiera se conectó)
- Cambios de variables de entorno en Vercel / Render / .env local
- Deploys manuales (ni Vercel ni Render)
- Envíos de WhatsApp (ni el helper CLI se corrió)
- Envíos de email
- Stripe API queries (no hay STRIPE_SECRET_KEY local)
- *git push --force · git reset --hard · merge a main*

10. Recomendación · plan ordenado

Paso	Acción	Riesgo
1 · INMEDIATO	Recuperar acceso del usuario afectado con el procedimiento manual (sección 4).	Caso si no código nuevo existe.
2 · Diagnóstico opcional	Verificar en Stripe que el caso sea efectivamente duplicado de customer (no <i>email_duplicate</i> ni secret rotado).	General read-only.
3 · Mergear hotfix v2	Tras tu OK, abrir PR de <i><i>hotfix/auth-cross-customer-recovery</i></i> . Auto-deploy con <i>git revert</i>	Bajo nivel.
4 · Post-deploy	Smoke login del usuario afectado con su password viejo. Esperado: entra al <i>Cashboard</i> verificando MW. Logs deben mostrar <i>login</i> .	Cashboard verificando MW.
5 · Housekeeping (PR aparte)	Consolidar duplicados de customer en Stripe (vía Customer Portal del usuario o <i>manage_customer</i> de Dashboard). Reduce fut.	Comando de Stripe Dashboard).

Próximo paso: tu decides si (a) solo recuperamos al usuario manualmente y dejamos v2 en draft para revisión, o (b) recuperamos manualmente Y mergeamos v2 para automatizar el caso. **Sin tu OK no se mergea ni se**

ejecuta el helper.